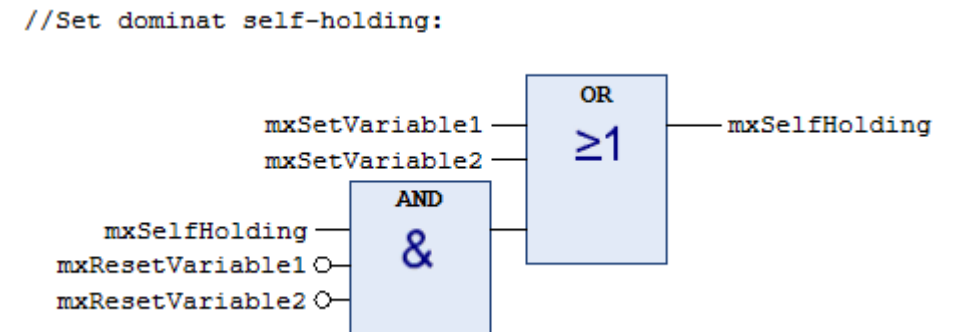
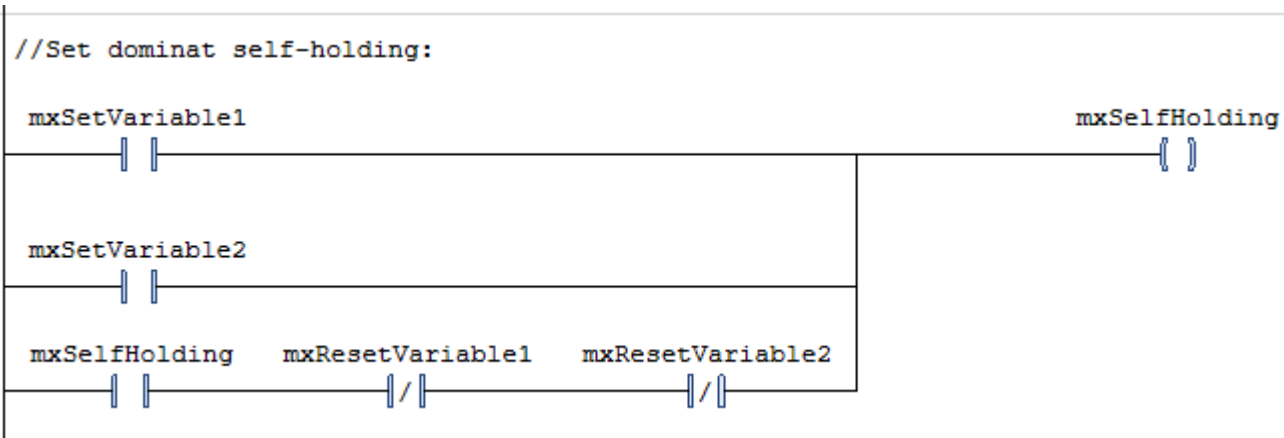
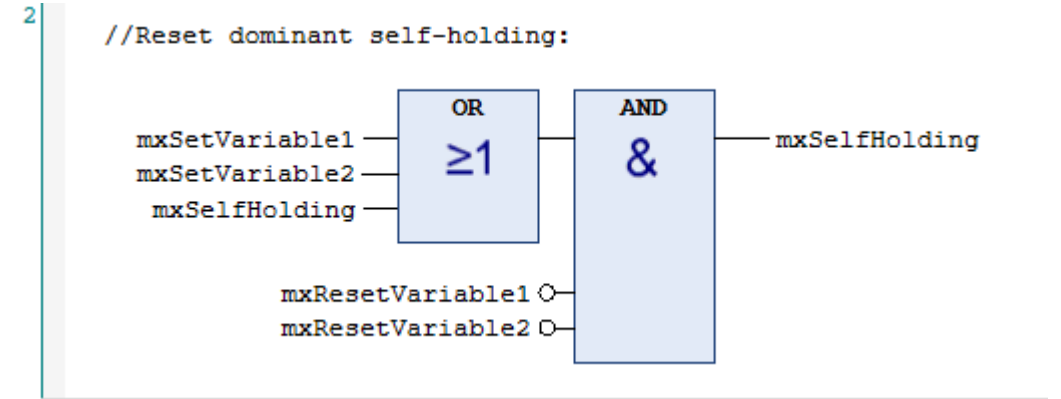
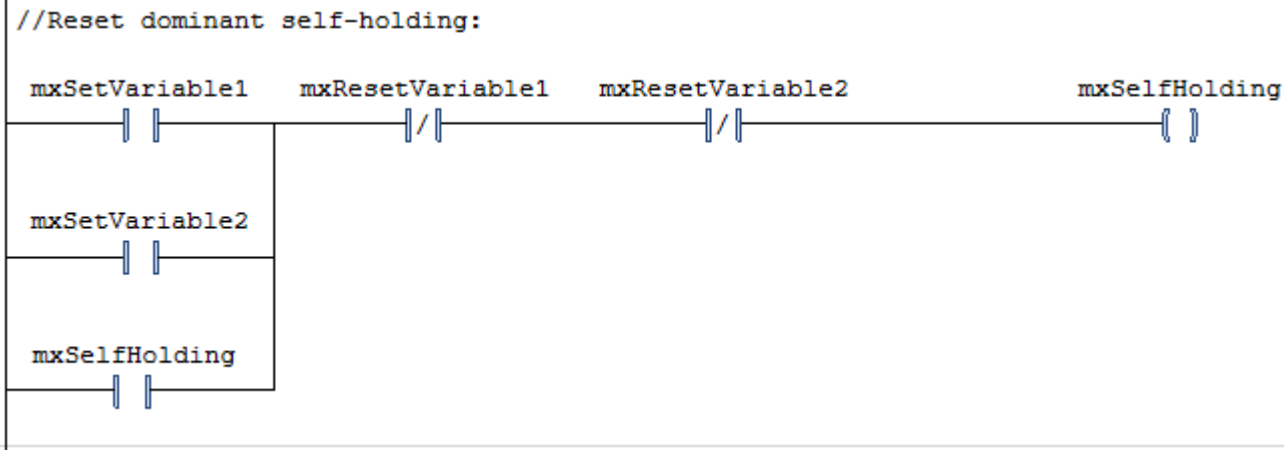


Lecture 4

Calculate and Compare

- Set and Reset dominant self-holdings il LD and FBD
- Number Systems
- Variable Types
- AD/DA
- Calculations
- Creation of functions and function blocks
- Creation of data types DUT

Set and Reset dominant self-holding



The Binary Number System

65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
Data Word = 16 bits									Byte=8bits				Nibble			
MSB													LSB			

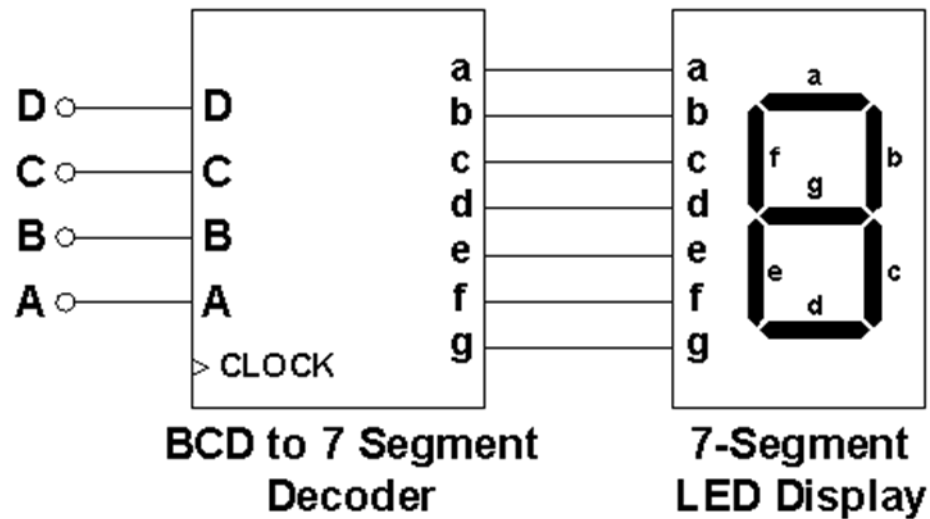
Bin	Dec
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11

The Hexadecimal number system

Binary	Decimal	Hexadecimal
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F
1 0000	16	10

Use 4-bits for each figure
Base 16

BCD-Coded, Hexadecimal per figure



Bin	Oct
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

The Octal number system

Decimal	Octal
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	10
9	11
10	12
11	13
12	14
13	15
14	16
15	17
16	20

- 3-Bits for each figure 0-7
- Base 8
- Used for I/O addresses

Convert between different number system

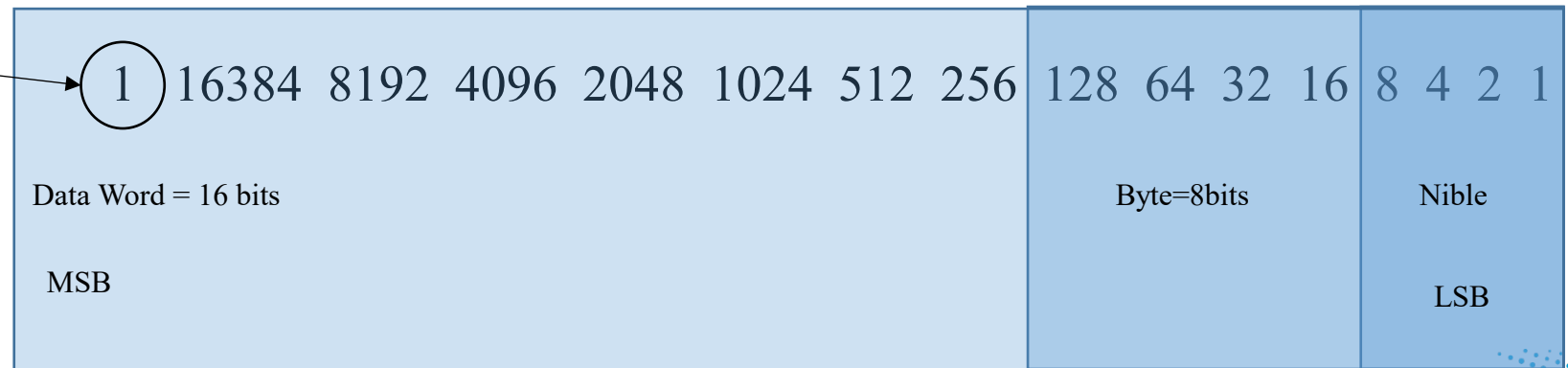
- Ex. Decimal (10) 3567 can be expressed
 $3 \cdot 10^3 + 5 \cdot 10^2 + 6 \cdot 10^1 + 7 \cdot 10^0 = 3567$
- Convert Binary (2) 1011 to Decimal
 $1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11$
- Hexadecimal (16) A2C5 to Decimal
 $10 \cdot 16^3 + 2 \cdot 16^2 + 12 \cdot 16^1 + 5 \cdot 10^0 = 41669$
 ↑
 (A Hex = 10 Dec)

Common integer Variable Types

- | | | |
|---------------------|----------|---------------------------|
| • BOOL (ix, qx, mx) | 1-bit, | 0 to 1 (FALSE to TRUE) |
| • WORD (iw, qw, mw) | 16-bits, | 0 to 65535 |
| • INT (mi) | 16-bits, | -32768 to 32767 |
| • DINT (md) | 32-bits, | -2147483648 to 2147483648 |

The minus sign is stored in the most significant bit of an **INT**

OBS!!! Turns **negative** if the value is more than **32767**



IEC 61131-3 integer data types

Data type	Lower bound	Upper bound	Memory space
BYTE	0	255	8 bit
WORD	0	65535	16 bit
DWORD	0	4294967295	32 bit
LWORD	0	$2^{64}-1$	64 bit
SINT	-128	127	8 bit
USINT	0	255	8 bit
INT	-32768	32767	16 bit
UINT	0	65535	16 bit
DINT	-2147483648	2147483647	32 bit
UDINT	0	4294967295	32 bit
LINT	-2^{63}	$2^{63}-1$	64 bit
ULINT	0	$2^{64}-1$	64 bit

From: https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_plc_intro/2529415819.html&id=

Numeric Constants

14	Decimal number
2#1001_0011	Binary number
8#67	Octal number
16#A	Hexadecimal number
DINT#16#A1	Typed data type DINT# and base 16# combined.

Select one bit of a WORD

VAR

mxMyBitVariable : BOOL;

mwMyWordVariable : WORD;

END_VAR

mwMyWordVariable := 3; // 2#0011

mxMyBitVariable := mwMyWordVariable.0; // mxMyBitVariable =1

mxMyBitVariable := mwMyWordVariable.1; // mxMyBitVariable =1

mxMyBitVariable := mwMyWordVariable.2; // mxMyBitVariable =0

Pointers and address operator ADR

```
VAR
```

```
  //declaration of pointer pSample
```

```
  pSample : POINTER TO INT;
```

```
  //declaration of variables nVar1 and nVar2
```

```
  nVar1 : INT := 5;
```

```
  nVar2 : INT;
```

```
END_VAR
```

```
//pointer pSample is assigned to address of nVar1
```

```
pSample := ADR(nVar1);
```

```
//value 5 of nVar1 is assigned to variable nVar2 by dereferencing of pointer pSample
```

```
nVar2 := pSample^;
```

Real variables for floating point numbers

Data type	Lower bound	Upper bound	Memory space
REAL	-3.402823e+38	3.402823e+38	32 bit
LREAL	-1.7976931348623158e+308	1.7976931348623158e+308	64 bit

- Reals are convenient to use
 - Will never turn unhappily negative as INT does
 - Have no hard limited min and max value
 - The resolution is better in an LREAL than in a REAL
- Hard to compare two real, they will rarely be equal
 - `IF(fMyReal = 10.0) THEN`
- Explicit type cast of a UINT to REAL `fMyReal := REAL#3400;`

Variable Types DATE TIME

- TIME 32bits T#1h2m3s23ms
- LTIME 64bits LTIME#5m90s15ms542us15ns
- DATE 32bits D#2023-2-7

- DATE_AND_TIME 32bits DT#2020-2-7-12:55:1.234;
- TIME_OF_DAY 32bits TOD#12:3:4.567

Unit	Description
d	Day
h	Hour
m	Minute
s	Second
ms	Millisecond
us	Microsecond
ns	Nanosecond

More variable types on Beckhoff Infosys

https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_plc_intro/2529415819.html&id=

String variables

- CHAR 8bits 'a' //8-bits ASCII table
- WCHAR 16bits 'a' //16-bits ASCII table
- STRING 'This is a string' //default 80 characters long
 - sVar : **STRING(35)** := 'This is a String';
- WSTRING 'A string of WCHAR'

7-bits ASCII table

0-31 are control signs for use in communication

Ex, 2 start of text, 10=Line feed, 13 Carriage return

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	`
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of xmit	EOT	CTRL-D	36	24	\$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	2F	/	79	4F	O	111	6F	o
16	10	Data line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of xmit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	FS	CTRL-\	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	GS	CTRL-]	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	RS	CTRL-^	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	US	CTRL-~	63	3F	?	95	5F	_	127	7F	DEL

String constants

Useful in communication e.g., to a display

VAR CONSTANT

```
sConstA : STRING := 'Hello $n $r People'; // Hello  
                                           People
```

```
sConstB : STRING := 'Hello People $21'; // Hello People!  
END_VAR
```

String with \$-Code	Interpretation
\$<8-bit code>	8-bit code: Two-digit hexadecimal number that is interpreted according to ISO/IEC 8859-1.
'\$41'	A
'\$9A'	©
'\$40'	@
'\$0D'	Control character line break, corresponds to '\$R' Carriage return

String with \$-Code	Interpretation
'\$L', '\$I'	Control character line feed, corresponds to '\$0A'
'\$N', '\$n'	Control character new line, corresponds to '\$0A'
'\$P', '\$p'	Control character form feed
'\$R', '\$r'	Control character line break, corresponds to '\$0D'
'\$T', '\$t'	Control character tab
'\$\$'	Dollar sign: \$
'\$"	Single quotation mark: '

Arrays

- <variable name> : ARRAY[<dimension>] OF <data type> (:= <initialization>);

VAR

```
myArraOfInt : ARRAY[0..9] OF INT := [0, 10, 20, 30, 40, 50, 60, 70, 80, 90];
```

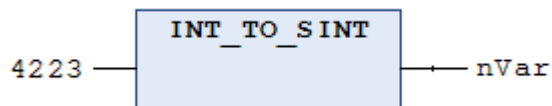
END_VAR

```
miMyInteger := myArraOfInt[2]; //Assign the value of 20 to miMyInteger
```

- Arrays can be of any data type, ex. ARRAY OF REAL or ARRAY OF STRING
- Arrays can consist of “arrays of arrays “ (in two, three, and more dimensions)

Convert between variable types

- nVar := INT_TO_SINT(4223);



- <INT type> TO <INT type>
- <type> TO BOOL
- BOOL TO <type>
- DATE/DT TO <type>
- REAL/LREAL TO <type>
- STRING TO <type>
- TO STRING/TO WSTRING for enumeration variables
- TIME/TOD TO <type>
- TO <type>
- TRUNC
- TRUNC_INT

Input outputs and memory addresses

I	Input memory area ("Inputs", physical inputs via input drivers, "Sensors")
Q	Output memory area ("Outputs", physical outputs via output drivers, "Actuators")
M	Flag memory area

X	Single bit		
B	Byte (8 bits)	%QX7.5	Output bit 7.5
		%Q7.5	
W	Word (16 bits)	%IW215	Input word 215
D	Double word (32 bits)	%QB7	Output byte 7
		%MD48	Double word at memory location 48 in the flag area.
		%IW2.5.7.1	The interpretation depends on the current controller configuration (see below)
		nVar AT %IW0: WORD;	Example of a variable declaration with address specification AT declaration

If the placeholders * (%I*, %Q* or %M*) are used, TwinCAT automatically performs addressing

Naming conventions

IEC61131-3

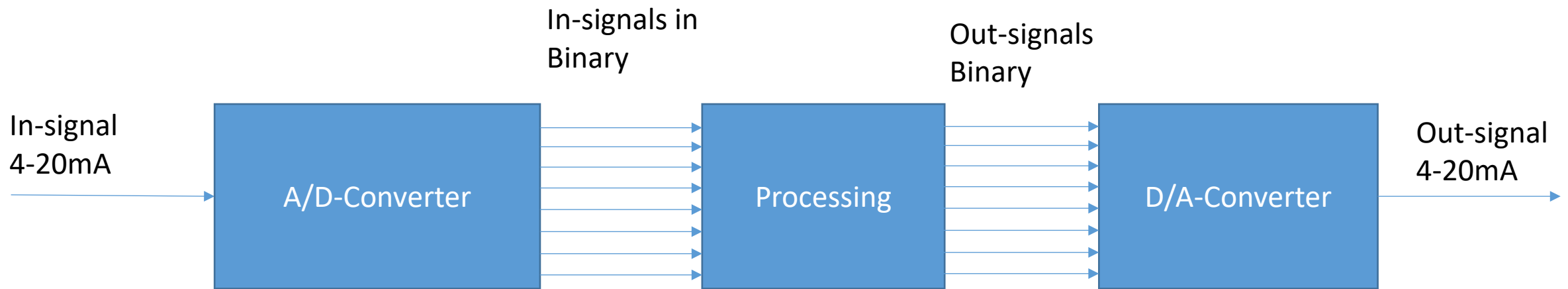
- ixMyInputBool qxMyOutputBool mxMyBoolean
- ibMyInputByte qbMyOutputByte mbMyByte
- iwMyInputWord qwMyInputWord mwMyWord
- miMyIntegerVariable
- mdMyDoubleIntegerVariable
- mrMyRealVariable

Beckhoff

- bMyBoolean
- nMyNumericVariable
- fMyFloatVariable

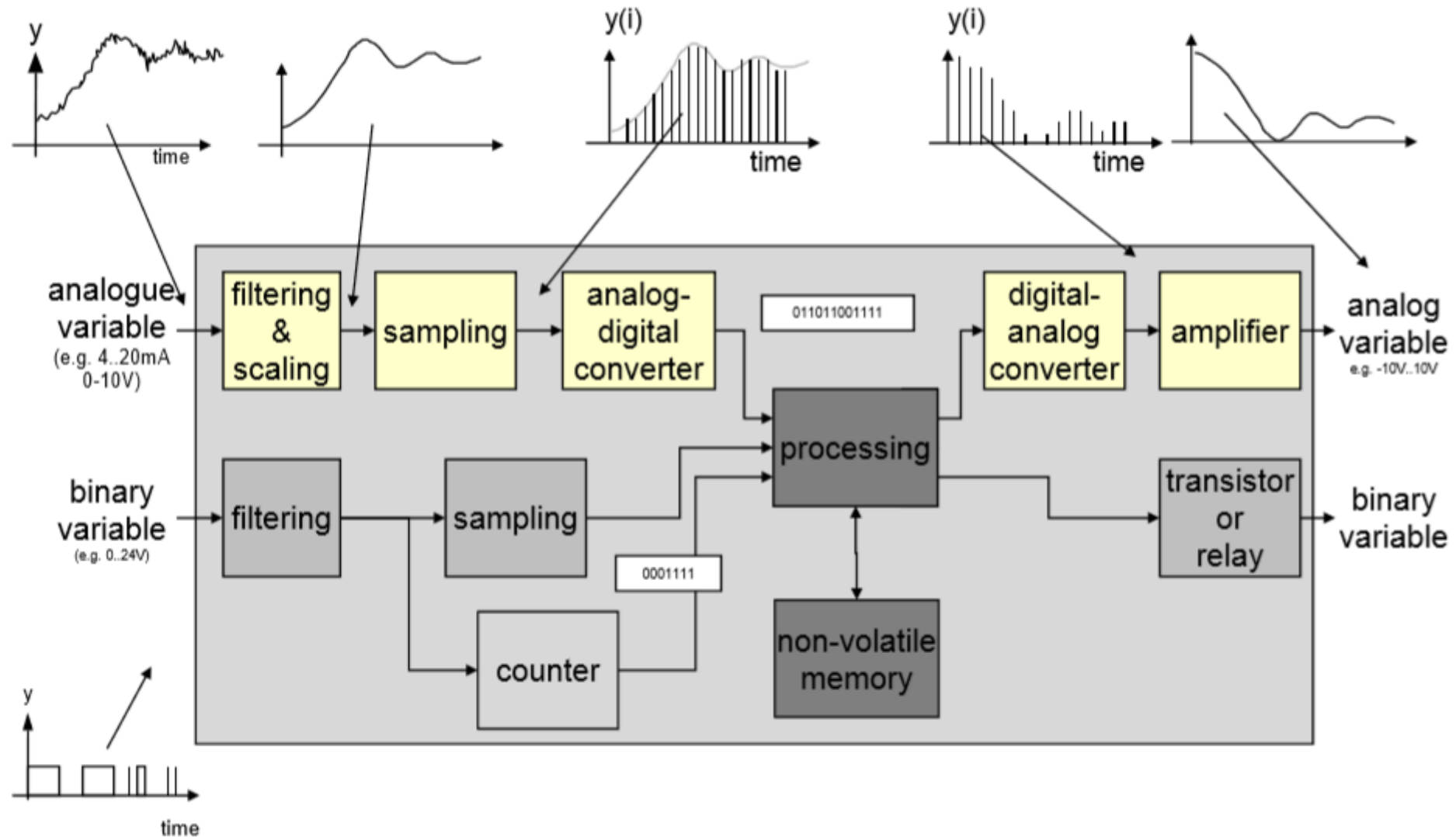
Use the IEC61131-3 naming convention!

A/D and D/A converter



A/D Analog to Digital
D/A Digital to Analog

The signal chain within a PLC

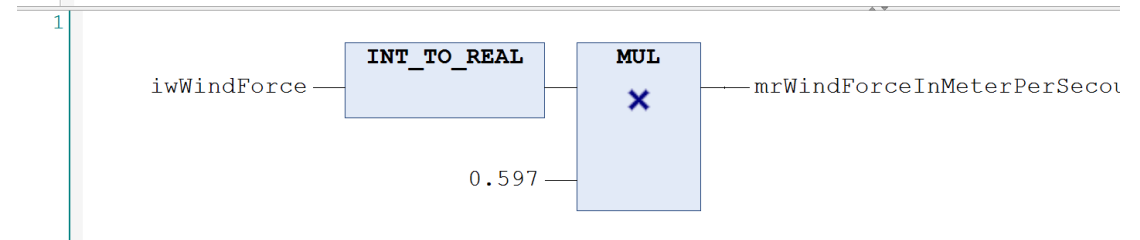


Declaration of an analog in-signal

iwWindForce **AT %I*:** **WORD**; (* 12-bits A/D value 0 to 4096
 stores in a 16 bits word *)

- Scaling is usually done in the sensor or in the application code or in the input device
- For some reason, 12-bit A/D converters are common
- Can be scaled by multiplying with a scale factor

```
1 PROGRAM POUtbd
2 VAR
3
4     mrWindForceInMeterPerSecound: REAL;
5     iwWindForce AT %I*: INT;
6 END_VAR
7
```



//Example of scaling

mrWindForceInMeterPerSecound := **WORD_TO_REAL**(iwWindForce) * 0.597

Declare as RETAIN to store the value after restart

```
VAR RETAIN
```

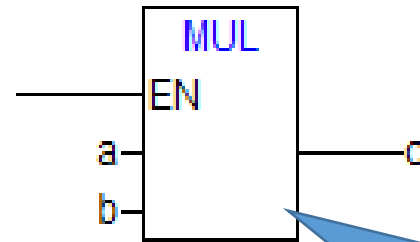
```
    miNumberOfRevolutions: INT;
```

```
END_VAR
```

Calculation and Compares

+	ADD
-	SUB
/	DIV
*	MUL

>	GT
<	LT
=	EN
≥	GE
≤	LE
≠	NE

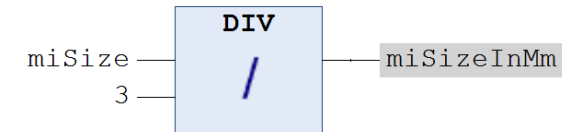


$c := a * b;$

Multiply a by b and store in c when EN is TRUE

Functions and Function Blocks in IEC61131-3

- AND, OR, MUL, DIV, GT, LE are all functions
- TON, CTU are function blocks
- Function blocks have names and are declared

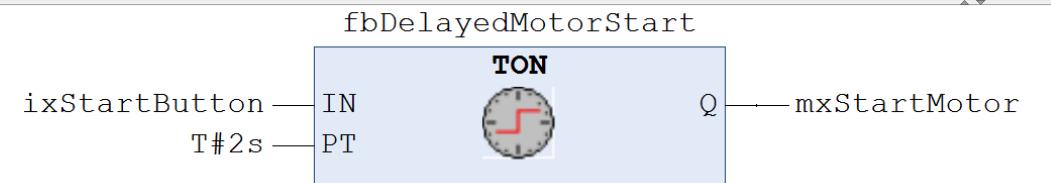


```
fbDelayedMotorStart.PT := T#2s;  
fbDelayedMotorStart.IN := ixStartButton;  
fbDelayedMotorStart();  
mxStartMotor := fbDelayedMotorStart.Q
```

VAR

```
fbDelayedMotorStart: TON;  
ixStartButton AT %I*: BOOL;  
mxStartMotor: BOOL;
```

END_VAR



Create a new reusable Function

- Right-click on POU
- Add POU
- Name the new function SCALE
- Select Structure Text
- Click Open

Add POU

Create a new POU (Program Organization Unit)

Name:
SCALE

Type

☐ Program

☐ Function Block

☐ Extends: ...

☐ Implements: ...

☐ Final ☐ Abstract

Access specifier:
...

Method implementation language:
Structured Text (ST)

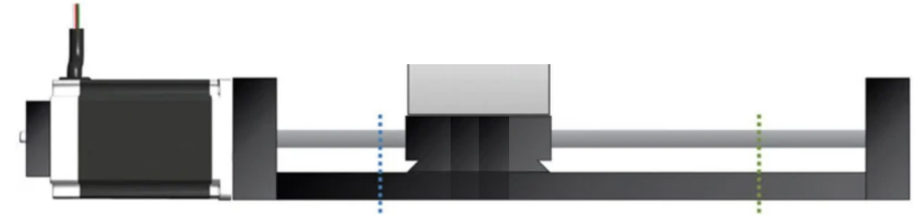
☒ Function

Return type: REAL ...

Implementation language:
Structured Text (ST)

Open Cancel

Create a new function SCALE



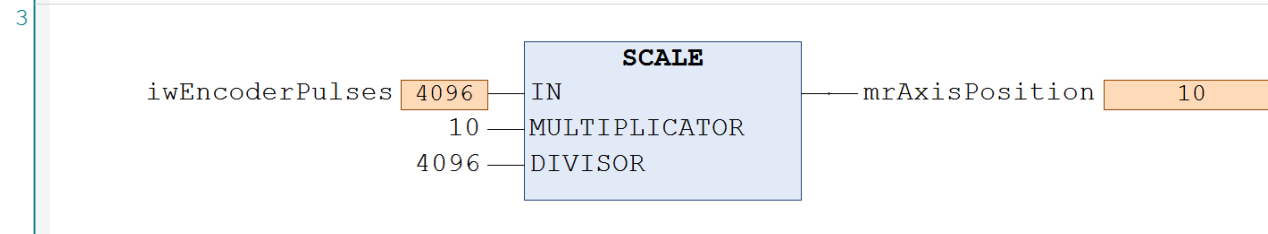
- Scales an adjusting axis from pulses to mm
- Encoder 4096 pulses per revolution
- Thread + Gearbox 10mm per revolution

◆ iwEncoderPulses	WORD	4096	%I*
◆ mrAxisPosition	REAL	10	

```

1 FUNCTION SCALE : REAL
2 VAR_INPUT
3     IN : INT;
4     MULTIPLICATOR : INT;
5     DIVISOR : INT;
6 END_VAR
7 VAR
8 END_VAR

```



```

1
2
3 SCALE := INT_TO_REAL(IN) * INT_TO_REAL(MULTIPLICATOR) / INT_TO_REAL(DIVISOR);

```

Creation of the Function Block BLINK

Add POU

Create a new POU (Program Organization Unit)

Name:
BLINK

Type

☐ Program

☒ **Function Block**

☐ Extends: ...

☐ Implements: ...

☐ Final ☐ Abstract

Access specifier:
 ▾

Method implementation language:
Structured Text (ST) ▾

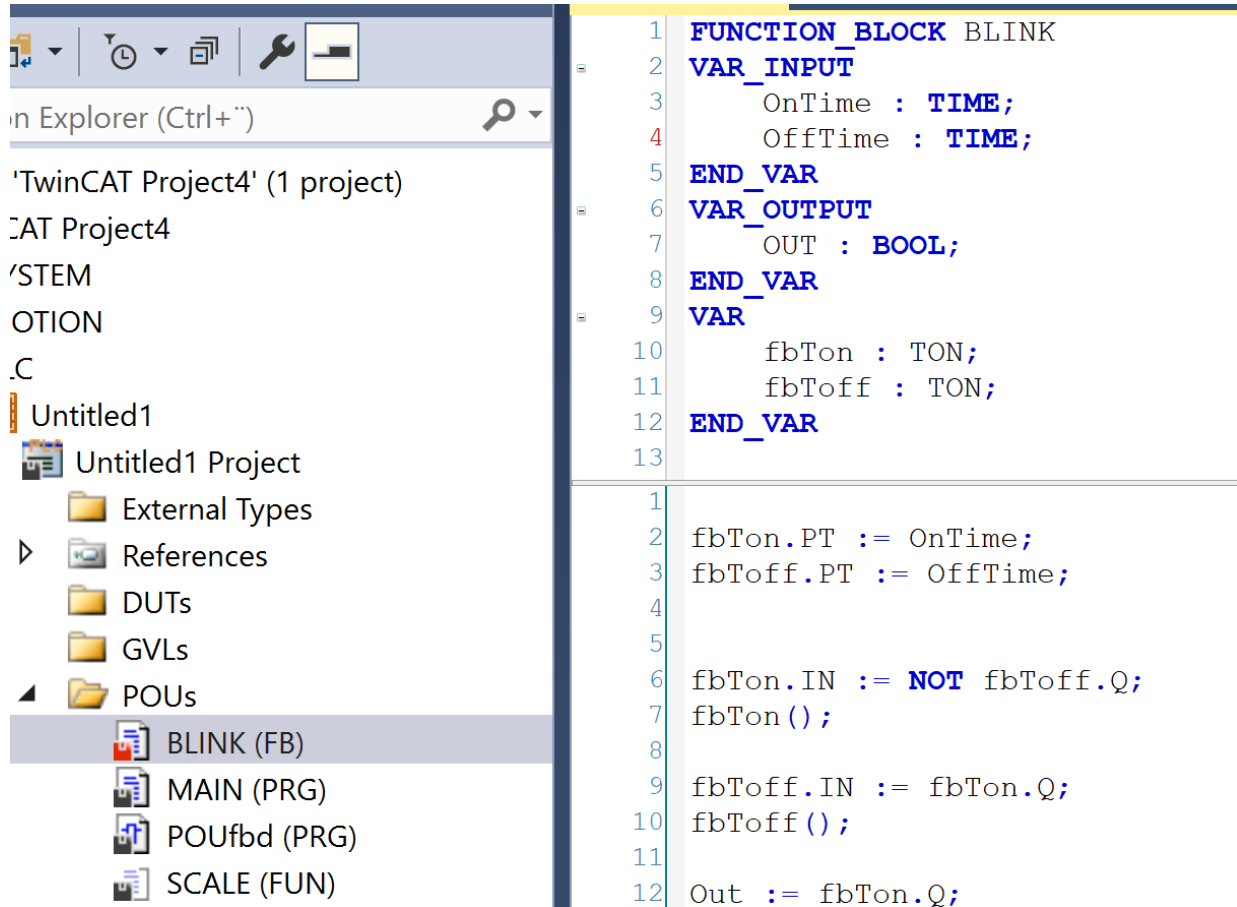
☐ Function

Return type: REAL ...

Implementation language:
Structured Text (ST) ▾

Open Cancel

Blink Function Block



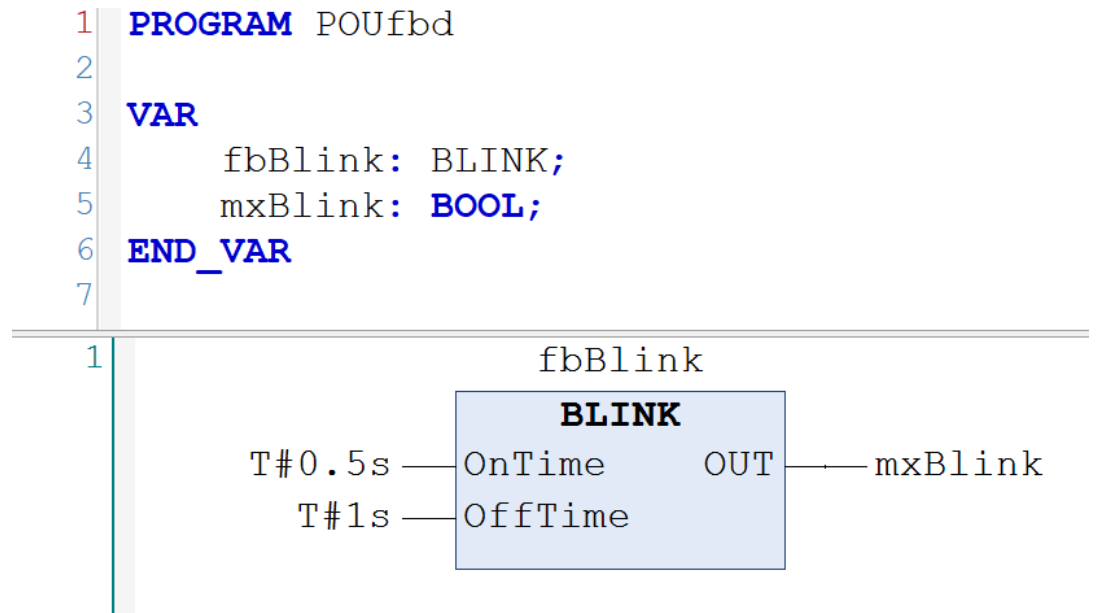
The screenshot shows the TwinCAT software interface. On the left is the Project Explorer with the following structure:

- 'TwinCAT Project4' (1 project)
 - CAT Project4
 - 'STEM
 - OTION
 - .C
 - Untitled1
 - Untitled1 Project
 - External Types
 - References
 - DUTs
 - GVLs
 - POUs
 - BLINK (FB)**
 - MAIN (PRG)
 - POUfbd (PRG)
 - SCALE (FUN)

The main editor displays the ladder logic for the BLINK function block:

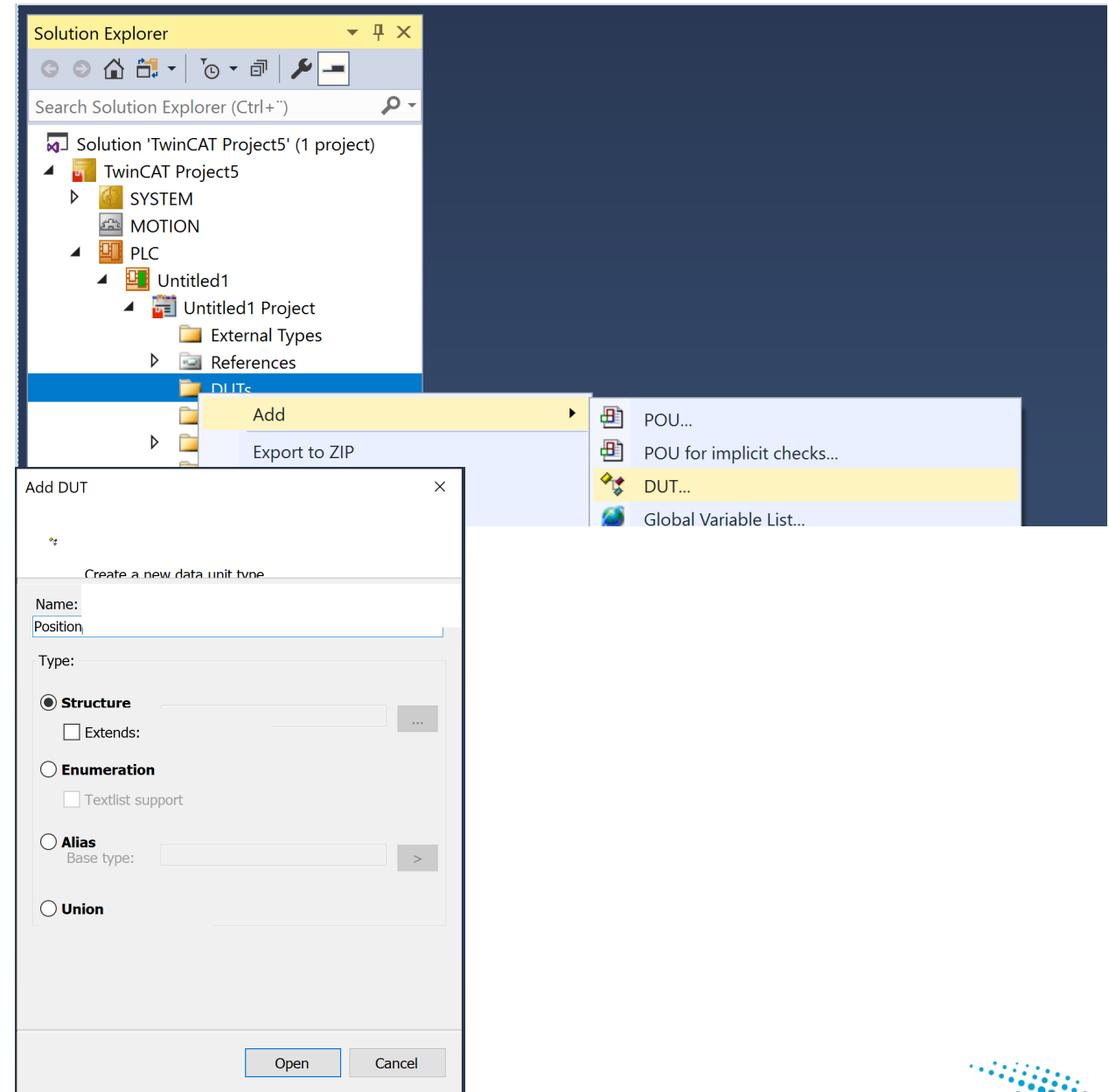
```
1 FUNCTION_BLOCK BLINK
2 VAR_INPUT
3   OnTime : TIME;
4   OffTime : TIME;
5 END_VAR
6 VAR_OUTPUT
7   OUT : BOOL;
8 END_VAR
9 VAR
10  fbTon : TON;
11  fbToff : TON;
12 END_VAR
13
```

```
1 fbTon.PT := OnTime;
2 fbToff.PT := OffTime;
3
4
5
6 fbTon.IN := NOT fbToff.Q;
7 fbTon();
8
9 fbToff.IN := fbTon.Q;
10 fbToff();
11
12 Out := fbTon.Q;
```



Data Unit Type (DUT)

- Right-click on DUTs
- Add DUT
- Select Structure
- Give the structure a name
- Use enumeration to give digits names



Example, 3-dimensional position of three REAL variables

```
Position*  X
1  TYPE Position :
2  STRUCT
3      x : REAL;
4      y : REAL;
5      z : REAL;
6  END_STRUCT
7  END_TYPE
8  |
```

DUT struct position variable

```
MAIN*  Position*
1  PROGRAM MAIN
2  VAR
3      pos: Position;
4      targetPos: Position;
5  END_VAR
```

- Two Position variables

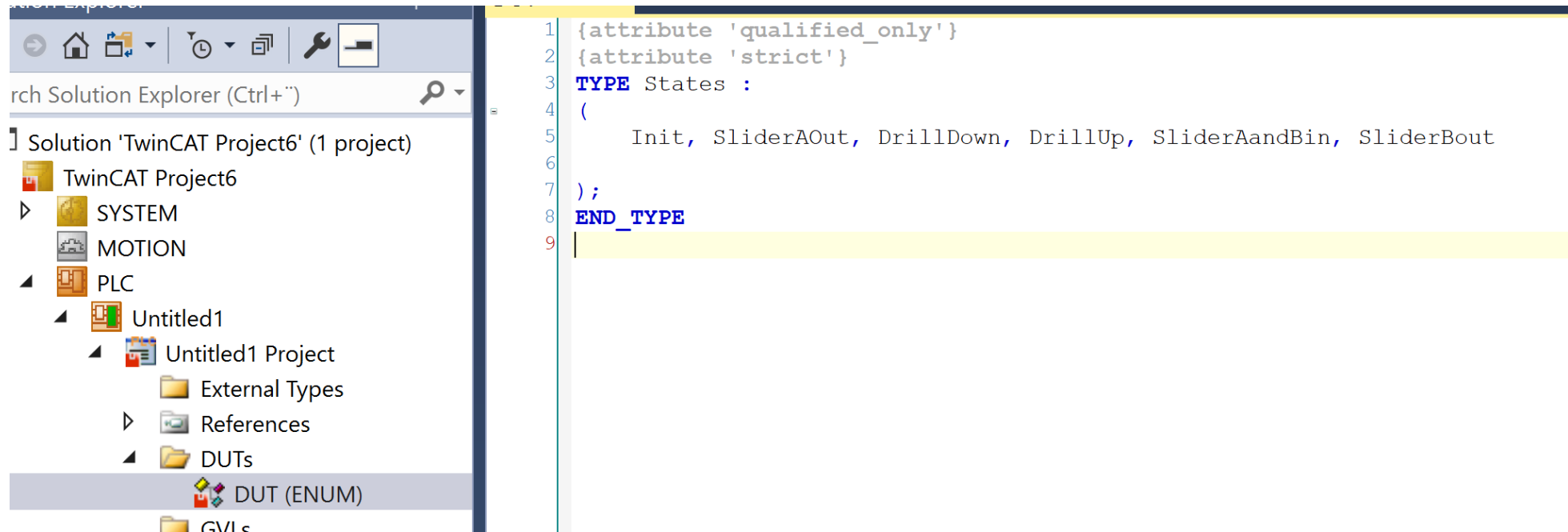
- Pos
- TargetPos

```
1  pos.x := 39.4;
2  pos.y := 300.0;
3  pos.z := 59.2;
4
5  targetPos := pos;
```

- Assign three coordinates

- Assign the whole pos variable to targetPos

DUT Enum



- special data type that enables a variable to be a set of predefined constants
- In this case Init = 0, SliderAout = 1,

Example of DUT Enum State

- Example of an Enum in a CASE
- State.Init: instead of 0:
- Facilitates adding of states in between

```
MAIN*  X  DUT
1 PROGRAM MAIN
2 VAR
3   miSequence: INT;
4   State: STATES;      //Init, SliderAOut, DrillDown, DrillUp, SliderAandBin, SliderBout
5
6 END_VAR

1
2
3
4 CASE miSequence OF
5
6   State.Init:
7     qxSliderAout := TRUE;
8     IF( mxStart) THEN
9       miSequence := State.SliderAOut;
10    END_IF
11
12   State.SliderAOut:
13     qxSliderAout := TRUE;
14     IF( ixSliderAout) THEN
15       miSequence := State.DrillDown;
16     END_IF
17
18   State.DrillDown:
19     qxSliderAout := FALSE;
20     qxDrillDown := TRUE;
21
22     IF( ixDrillDown) THEN
23       miSequence := State.DrillDown;
24     END_IF
25 END_CASE
```

Chapter 4 in Programmable Logic Controller

Dag H. Hansen

Structured Design

- 4-4.2 Number system
- Test Problems 4.3